Review plan · for the teacher

Teaching and assessing code: unit check

Programming · Level 3 · Unit check

Time	Materials
45 min	Board, chalk, paper. No devices needed.

Learning goals

- Read before you write: PRIMM
- Parsons problems
- Teaching debugging
- Misconceptions and feedback

Activity: Team quiz

1. Make teams of four
2. Read each question aloud; teams agree on one answer
3. For each wrong option, name the misconception behind it
4. Write the hardest question on the board for tomorrow

Discussion

- Which technique will you use first, and with which class?
- Which question surprised you, and why?
- Which misconception will you look for next week?

Exit ticket

- One change I will make to how I teach programming

Try it at school tomorrow
Pick one technique from this course (a PRIMM predict question, Parsons strips, the debugging routine or a hinge question) and use it in your next programming lesson. Then tell a colleague what the pupils' answers showed.

Quiz sheet · for each team

Teaching and assessing code: unit check

Programming · Level 3 · Unit check

Team _____ Date _____

1 **Which PRIMM stage is "Change the program so it counts in twos"?**

◯ Investigate ◯ Predict ◯ Modify ◯ Make

2 **A good Parsons problem has exactly one correct order.**

◯ True ◯ False

3 **Put the lines in order: the program prints the class average. One line doesn't belong.**

Number the lines in order (1, 2, 3…). Cross out any line that doesn't belong.

☐ `total = sum(marks)`

☐ `marks = [12, 15, 9]`

☐ `print("Average:", average)`

☐ `average = total / 2`

☐ `average = total / len(marks)`

4 **This program should print 1 to 10. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  # print 1 to 10
2  for n in range(1, 10):
3      print(n)
```

5 **Match each idea to what the research says.**

PRIMM	• • • •	Ordering lines takes less time than writing, with similar learning
Parsons problems	• • • •	A driver and a navigator swap roles often
Hinge questions	• • • •	Predict and run before you modify and make
Pair programming	• • • •	Each wrong answer points to a misconception

6 **A hinge question. What does this program print?**

```
x = "2"
print(x * 3)
```

◯ 222 ◯ An error ◯ 6

7 **Which of these are common programming misconceptions?**

☐ range(1, 5) includes 5 ☐ = and == mean the same

☐ Python runs the lines from the top down ☐ input() gives a number

8 **Fill the gaps so the program says Weekend! only on Saturday and Sunday.**

```
day = input("Day? ")
if day == "Sat" _____ _____ == "Sun":
    print("Weekend!")
```

Word bank: `and` `"day"` `or` `day`

9 **Put the debugging routine in order.**

☐ Check your guess by printing values

☐ Change one thing and test again

☐ Read the error message

☐ Find the line, and the one before

10 **Fix this program: it should print the cost, like Total: 150, but it prints something very strange.**

```
price = int(input("Price? "))
count = input("How many? ")
print("Total:", price * count)
```

Write your program:

11 **Asking a stuck pupil "What should total be after the first time round?" builds their debugging skill more than fixing the line for them.**

◯ True ◯ False

Answers · for the teacher

Teaching and assessing code: unit check

Programming · Level 3 · Unit check

1 **Modify**

Changing a working program to do something new is Modify.

2 **True**

If two lines could swap, a pupil could build a correct program and be marked wrong.

3 **Top to bottom: 2, 1, 4, X, 3**

```
marks = [12, 15, 9]
total = sum(marks)
average = total / len(marks)
print("Average:", average)
```

Each line needs the one before. Dividing by a fixed 2 instead of len(marks) is a hard-coding mistake: it only works for one list.

4 **Line 2: `for n in range(1, 11):`**

range stops before its second number, so it prints 1 to 9.

5 **PRIMM — Predict and run before you modify and make**
Parsons problems — Ordering lines takes less time than writing, with similar learning
Hinge questions — Each wrong answer points to a misconception
Pair programming — A driver and a navigator swap roles often

Read before writing, order before writing, find misconceptions quickly, and share the thinking: the four ideas behind this course.

6 **222**

x is text, and text times 3 repeats it: 222.

7 **= and == mean the same; range(1, 5) includes 5; input() gives a number**

The first three are classic misconceptions.

8

```
day = input("Day? ")
if day == "Sat" or day == "Sun":
    print("Weekend!")
```

Each side of or must be a whole condition: day == "Sun".

9 **Top to bottom: 3, 4, 1, 2**

Read, locate, check, then change one thing.

10

```
price = int(input("Price? "))
count = int(input("How many? "))
print("Total:", price * count)
```

count is text, so price * count repeats it instead of multiplying: a logic error with no message.

11 **True**

A question sends the pupil back into their own code and teaches the move they need next time.